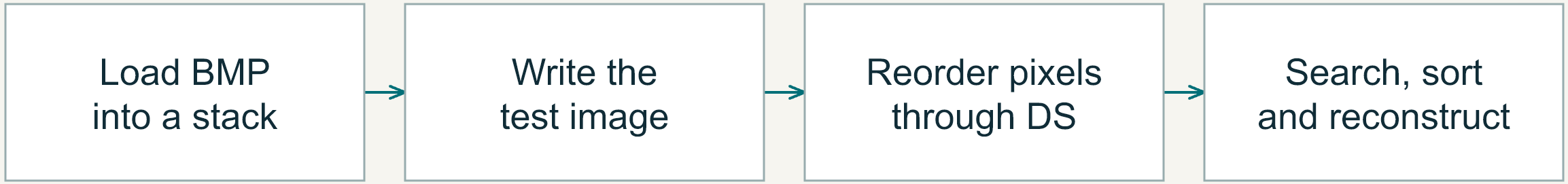
BMP image processing with linear data structures

Kaito Snezhko and Ana Gasperov
Data Structures · Lab Assignment 1 · UAH 2026–2027

Four stages of image processing



Input: RGB pixels, image width and image height

Outputs: _test.bmp, _stage3.bmp, _sorted_R/G/B.bmp, _flop.bmp

ADT selection and operations

StackDS and QueueDS

push(Pixel): void
pop(): Pixel
top(): Pixel
isEmpty(): bool

enqueue(Pixel): void
dequeue(): Pixel
isEmpty(): bool
getSize(): int

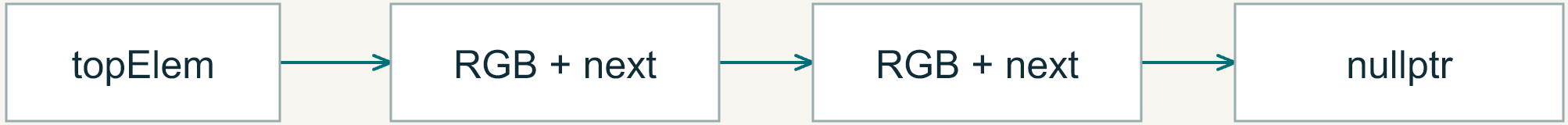
Rows and image

insertEnd(Pixel): void
insertSorted(Pixel, component): void

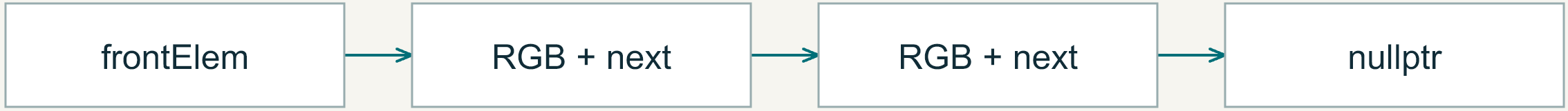
addRow(RowListDS*): void
getFirstRow(): RowListDS*

Linked stack and queue in memory

Stack: insertion and removal at the top



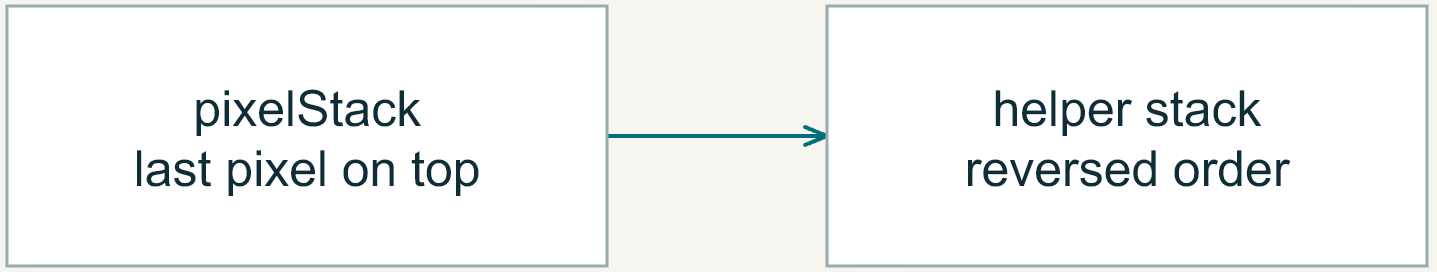
Queue: remove at front, insert at rear



rearElem points to the last pixel node

Each pixel stores RGB values and one next pointer

Stage 2 preserves the original stack



Copy back after writing

One pop writes one output pixel at a descending index

Moving every helper pixel back restores the stack

Result: the same image and the same stack contents

Stage 3 distributes pixels by priority

S1	i = 1 or i divisible by 2
S2	otherwise, i divisible by 3
S3	otherwise, i divisible by 5
S4	otherwise, i divisible by 7
S5	otherwise, i divisible by 11
S6	all remaining pixels

Example: i = 6 enters S1 because the first rule wins

Stage 3 merges the stacks into a queue

Queue position 1 and every even position take from S1.

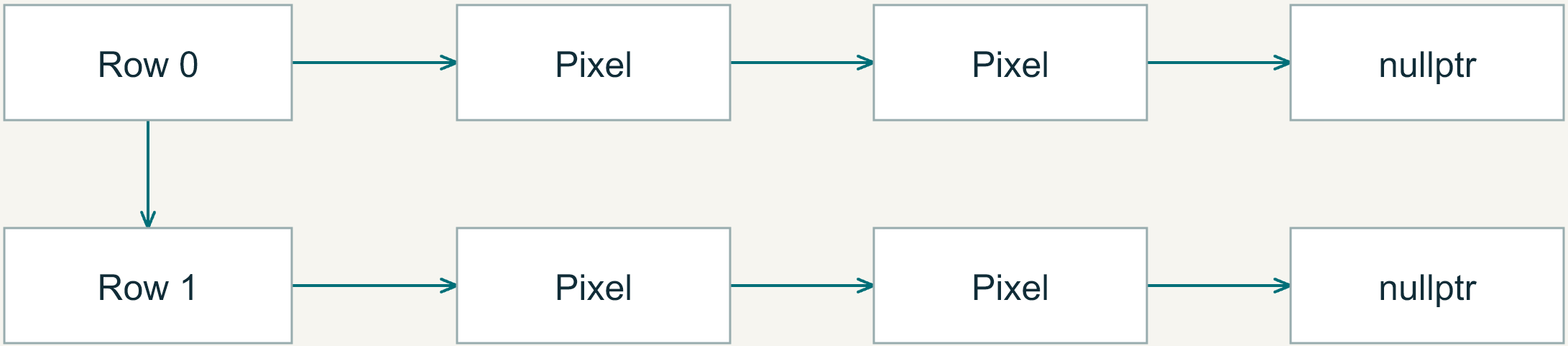
Other odd positions drain S6, then S5, S4, S3 and S2.

Queue positions



The queue preserves the new order for building image rows

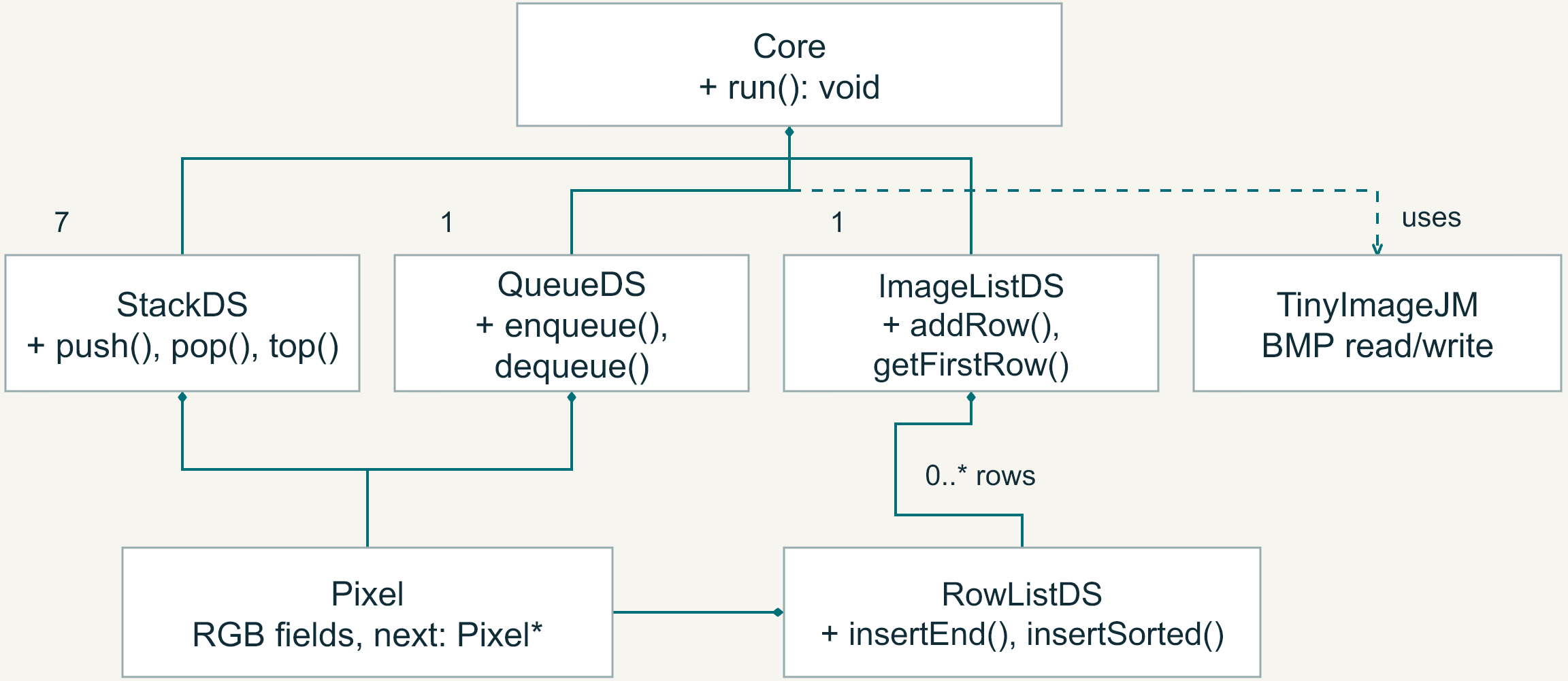
A list of lists represents the image



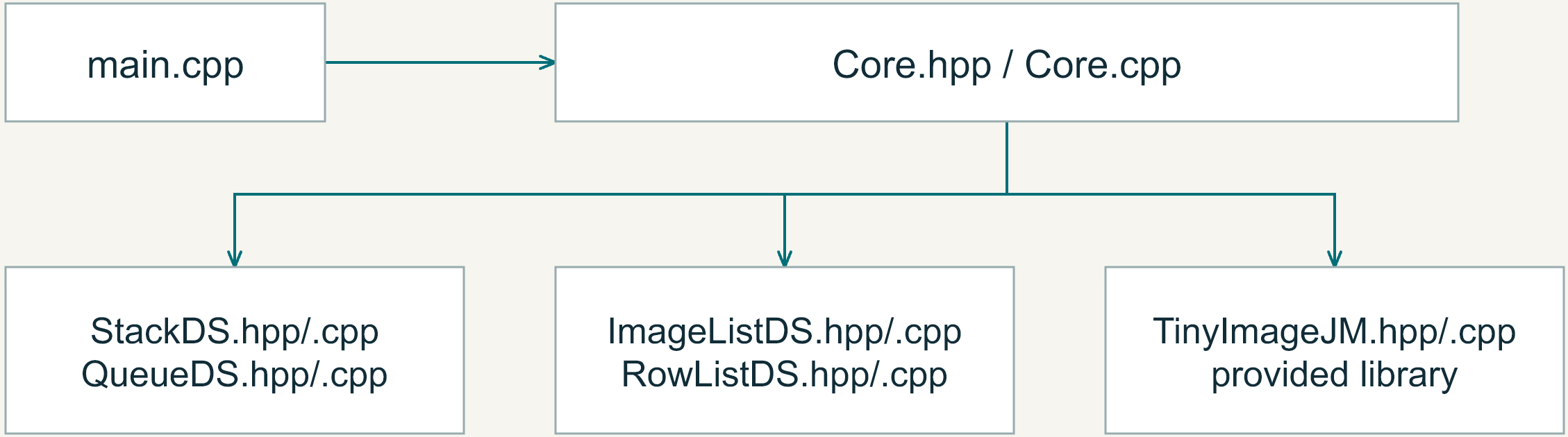
Each row contains W pixels. The image contains H rows.

Coordinates come from position, so nodes store only colour and links

Class diagram



Source files and dependencies



Pixel.hpp / Pixel.cpp supports the linked structures

Search by RGB colour

Input: red, green and blue values from 0 to 255

One traversal checks every pixel for an exact RGB match.

Output: compact (x, y) coordinates and the number of matches.

Coordinate conversion

x = position in the row
y = H − 1 − stored row index

Insertion sort builds a separate sorted row

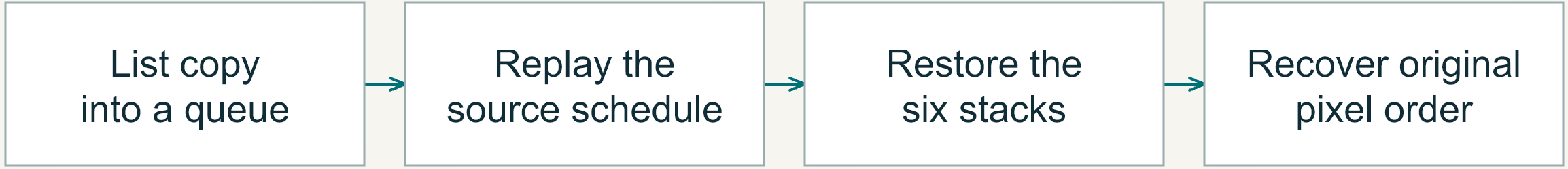
Example: selected red-channel values

Original row	90	20	60	20
Sorted copy	20	20	60	90

Ascending order by the selected component

Equal values keep their order. The source row stays unchanged.

Reconstruction reverses the permutation



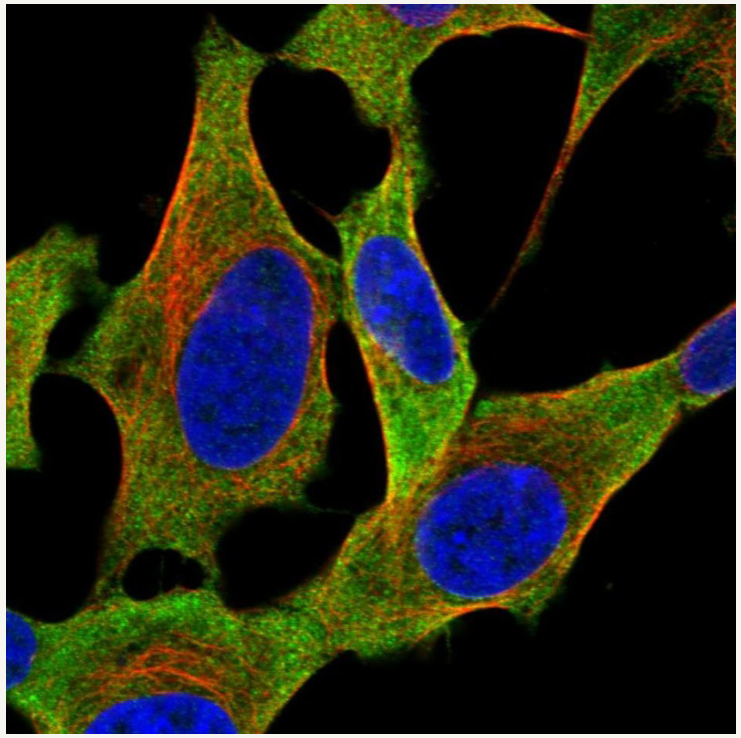
Horizontal mirror

(x, y) becomes (W − 1 − x, y)

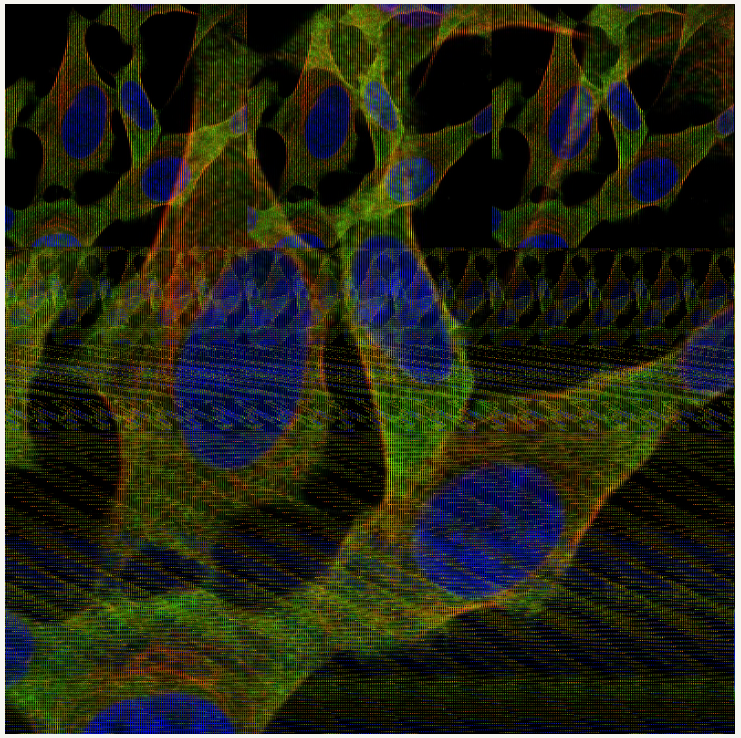
Temporary structures preserve the stored stage-3 image

Image outputs

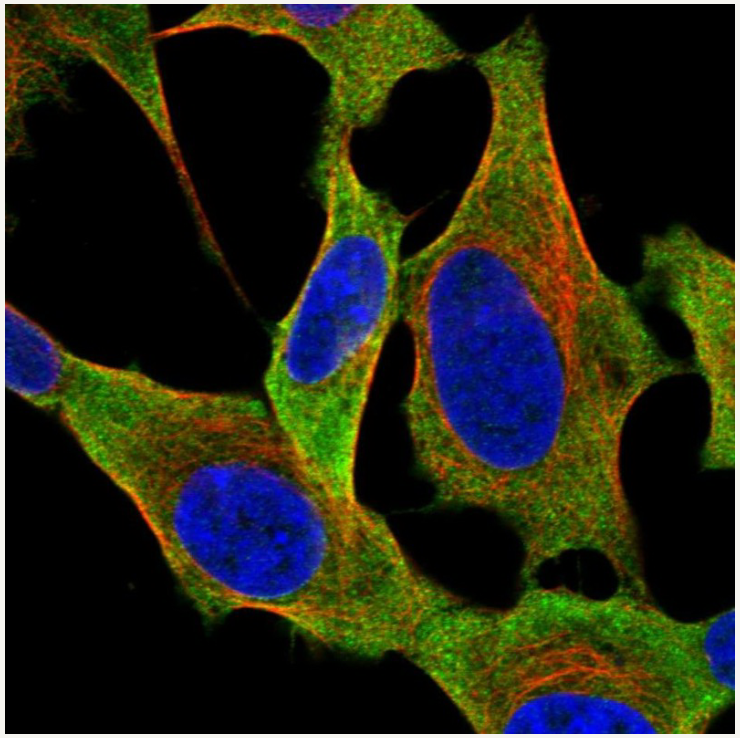
Original



After stage 3



Reconstructed + mirrored



Running time

W = width H = height N = W × H

Stack push/pop and queue enqueue/dequeue	**O(1)**
Automatic stages, search and reconstruction	**O(N)**
Row sorting, worst case	**$O(HW^2)$**

Already ascending rows use the tail shortcut: O(N) sorting

Dynamic storage and memory costs

Why linked structures fit this assignment

The node count follows the image size.

Stack and queue end operations need no pixel shifts.

Pointers and individual allocations add overhead.

Auxiliary storage

Sorting: O(W) temporary row
Reconstruction: O(N) temporary nodes

Checks on the saved image outputs

Three input sizes

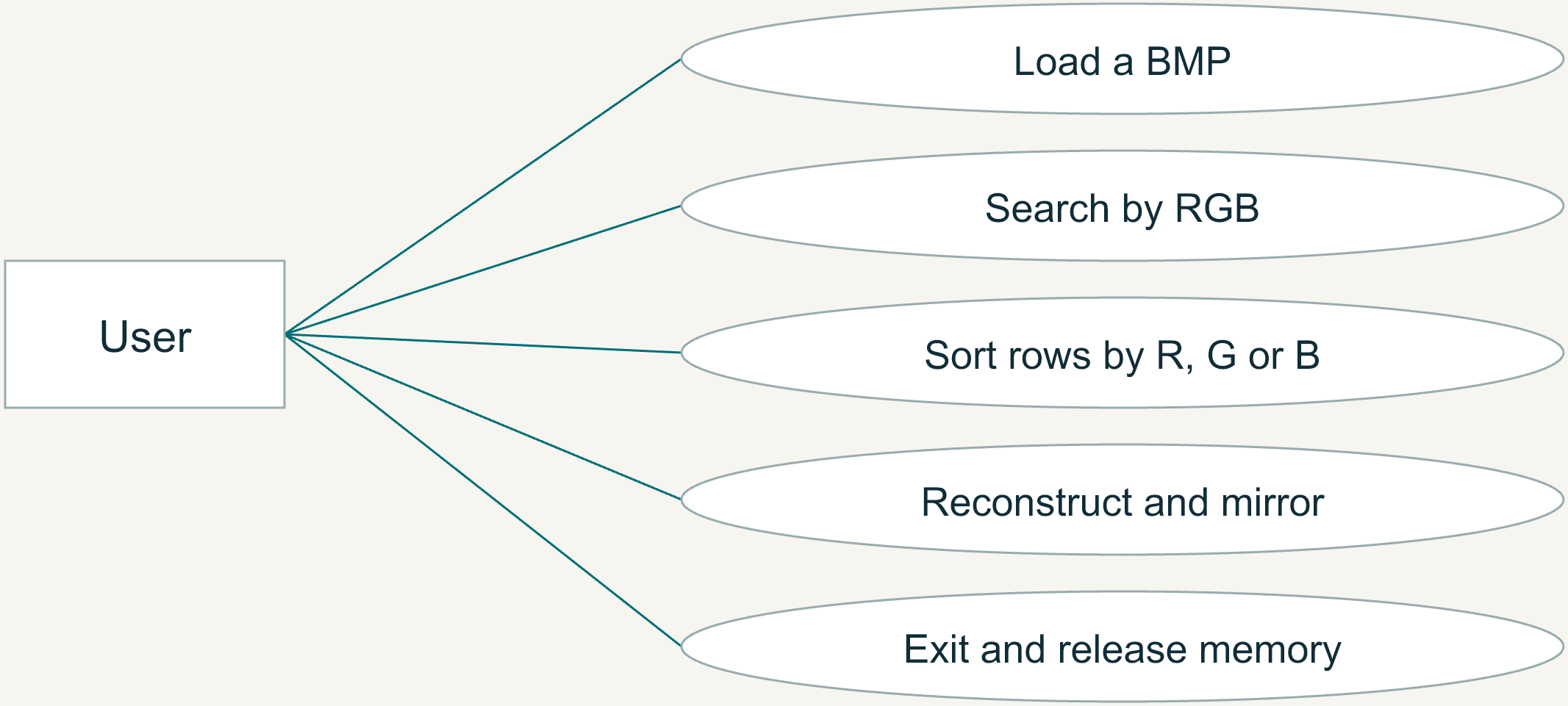
800 × 800 1108 × 1108 2000 × 1334

_test.bmp matches the original pixel for pixel.

_flop.bmp matches a horizontal mirror of the original.

Evidence: pixel comparisons of the existing saved output files

User operations



Conclusions and improvements

What the program demonstrates

Stacks and a queue create a reversible pixel permutation.

Temporary structures preserve data during menu operations.

Row insertion sort has the highest worst-case cost.

Further improvements

Check each image reload. Expand BMP edge-case tests.
Consider merge sort for wider rows if the specification allows it.

References

UAH Data Structures, Fall 2026 laboratory assignment

Presentation-session instructions and Group G feedback checklist

Project sources: Core, Pixel, StackDS, QueueDS, RowListDS, ImageListDS

Provided TinyImageJM library and project BMP test/output files